Advances in Cognitive Systems X (20XX) 1-16 Submitted X/20XX; published X/20XX

Hierarchical Compositionality for An Assistive AI Agent

Tianyi Fu TIANYI.FU@ED.AC.UK
Mohan Sridharan M.SRIDHARAN@ED.AC.UK
School of Informatics, University of Edinburgh, Edinburgh, EH8 9YL, United Kingdom

Abstract

AI agents are increasingly being developed to assist humans in various applications, and Large Language Models and other deep network architectures are considered state of the art for such agents. These methods are impressive stochastic predictors, but they are resource-hungry, opaque, and known to make arbitrary decisions in novel situations. Our work seeks to explore the design of architectures for such AI agents based on core principles that can be traced back to the early pioneers of AI but are not fully utilized in modern AI methods. We do so in this paper in the context of the core problem of an AI agent addressing ambiguity in the objects being referred to by the human participants. Humans address such ambiguity by heuristically leveraging compositional domain knowledge and the preferences of the other human participants. Inspired by this observation, we describe an architecture that embeds the principle of hierarchical compositionality and uses simple heuristics to achieve the desired disambiguation. Specifically, domain objects are represented in terms of primitive attributes drawn from human-validated semantic feature norms, and a hierarchical combination of *attributes* and *concepts* automatically identified from a limited observed history of interactions of an assistive agent with specific users. The assistive agent then achieves the desired disambiguation by reasoning with knowledge of this compositional hierarchy; axioms governing domain dynamics; and models of semantic compatibility, session salience, and user-specific thematic preference, requesting human clarification when necessary. Experiments show that our approach consistently outperforms state of the art data-driven baselines, supporting adaptation to specific user profiles. Project website: https://tianyi-fu.github.io/HCAA/

1. Motivation

We are witnessing a rapid proliferation of AI agents being deployed to assist humans in various applications. Deep networks and *foundation models* (FMs) such as Large Language Models (LLMs) and Vision Language Models (VLMs) are considered state of the art for designing such AI agents. Although they have been reported to provide impressive performance in perception, reasoning, and interaction problems, these methods are opaque and resource-hungry. The underlying stochastic prediction system captures the most likely patterns, which may lead to inconsistent and arbitrary responses when the AI agent has to quickly make personalized decisions in novel situations (Kambhampati et al., 2024).

Our work seeks to explore the design of architectures for such assistive AI agents based on fundamental principles such as iterative refinement (i.e., compositionality), ecological rationality, and interactive learning, which trace back to the early pioneers of AI but are not fully leveraged

© 20XX Cognitive Systems Foundation. All rights reserved.

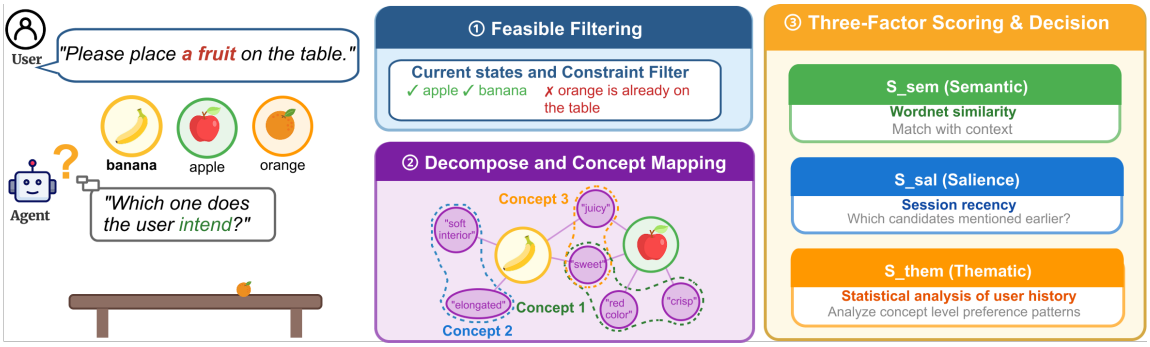


Figure 1. Architecture overview and illustrative example of an AI agent receiving an ambiguous request.

by modern AI methods (Sridharan, 2025). Also, humans use heuristics based on core capacities (e.g., to sum, order, imitate, recognize, forget, track) for robust decisions *in the wild*, i.e., under uncertainty or intractability (Gigerenzer, 2016). Our long-term objective is to determine if and how these fundamental principles should be introduced in architectures for assistive agents, with modern AI methods being one of many tools these agents use.

The representative use case considered in this paper is that of an AI agent that has to resolve the ambiguity in the objects being referred to by the human user, although disambiguation (a well-researched topic) itself is not the main focus of this paper or of our work. Consider such an agent that has been asked by a human to "place a fruit on the table" in a kitchen containing apples, oranges, and bananas, as shown in Figure 1. Such ambiguous statements are common in human conversations, and we address them using knowledge of the domain and of each other's preferences. It is challenging for AI agents based on deep networks to achieve such personalized disambiguation rapidly, and answering many clarification questions from the AI agent will be a cognitive burden for the human user. Some hybrid frameworks combining data-driven models and knowledge-based reasoning have also been developed to address ambiguity in task specification (Lin et al., 2025), but effective personalized interaction remains an open problem.

In designing our architecture, we specifically focus on embedding the principle of hierarchical compositionality and on using simple heuristics. Specifically, our architecture:

- Represents entities as hierarchical compositions of atomic attributes drawn from human-validated semantic feature norms and compound *concepts* automatically mined from these attributes, with personalized preferences of human users learned from observations.
- Computationally and heuristically models semantic compatibility, session salience, and user-specific thematic preference as complementary factors.
- Performs non-monotonic logical reasoning with current knowledge of concept hierarchy, axioms governing domain dynamics, and computed model of contextual information to achieve disambiguation, requesting clarification from humans only when needed.

We use Answer Set Programming (ASP) (Gelfond & Kahl, 2014) as the non-monotonic logical reasoning paradigm, and GPT-5.1 (Singh et al., 2025) as the LLM.

2. Related Work

2.1 Command Disambiguation and Personalization

With the increasing use of FMs in Human-Robot Interaction, prior work has addressed ambiguity in such models, e.g., prompting LLMs to generate clarification questions (Park et al., 2023) and detecting referential ambiguity from the cross-modal attention of a VLM (Abrini & Chetouani, 2026). However, these methods rely solely on information encoded in the FMs, visual features extracted from the scene, or dialogue history, without modeling individual user preferences. We instead draw inspiration from a framework that leverages the interplay between knowledge-based reasoning and data-driven learning (Mota & Sridharan, 2021). However, even this framework relies on substantial human supervision and lacks the hierarchical compositional representation of knowledge that is a key component of our architecture.

In practice, the same ambiguous command may refer to different entities for different users and in different contexts; even recent benchmarks assume no access to user history, deliberately imposing a zero-context setting in which every preference-type ambiguity must be resolved by asking the user (Ivanova et al., 2025). Approaches exploiting user-specific evidence either reuse past referring expressions recorded for each object (Roy et al., 2019) or retain explicitly stated preferences in an LLM memory (Abugurain & Park, 2024), but do not transfer the accumulated evidence across entities. Existing approaches also learn behavioral patterns from implicit signals such as command history (Fischer, 2001; Fu et al., 2025). However, there is a scarcity of data needed to acquire preferences at the (low) level of primitive actions and specific entities. Our architecture links user-specific preferences to the learned compositional concepts, enabling transfer across related entities from limited interaction history.

2.2 Compositional Hierarchical Representations

Hierarchical compositional representations have been studied for many decades in the context of human cognition (Fodor, 1975; Piantadosi et al., 2016). In visual recognition, low-level features were recursively composed into shared parts across object categories (Fidler & Leonardis, 2007), and shared semantic properties enabled zero-shot transfer to unseen categories (Lampert et al., 2013). In robotics, encoded and learned ontologies (and knowledge graphs) have been used for planning (Beetz et al., 2018), and motion primitives are composed through graph structures for novel tasks (Tian et al., 2024). This principle has also been applied to referring expressions in human-robot interaction (Gao et al., 2023).

Deep networks can generalize compositionally when the data contains learnable structure that the model can align with, and the training distribution covers the relevant combinations (Boopathy et al., 2024). They can often answer the individual sub-questions of a task but fail when the task requires composing them in a specific manner, and this *compositionality gap* does not necessarily close with additional data (Press et al., 2023). Concept-based models introduce a concept layer (Koh et al., 2020). Neurosymbolic methods combine neural perception with logical or probabilistic inference (Chen et al., 2025). These methods often rely on predefined concepts and concept-level annotations, shifting much of the modeling burden to humans and limiting adaptation to fine-grained, user-specific preferences in changing contexts. Our approach lies between fully implicit neural

representations and explicit symbolic representations. The basic set of attributes is chosen from an open-source dataset of human-validated features, but the concept-level structures above it are learned from attribute co-occurrence and user interaction history, capturing subtle preferences that are difficult to specify in advance. These structures can be audited and revised.

2.3 Decision Heuristics

Heuristics are often viewed as shortcuts, but the early pioneers of AI and modern mathematics drew inspiration from the use of heuristics by humans to propose strategies for solving intractable problems (Katsikopoulos & Gigerenzer, 2026). One example of a related computational theory is Herb Simon's work on *Bounded Rationality* and heuristics (Simon, 1956). Over time, bounded rationality has been incorrectly equated with maximizing expected utility (Russell & Norvig, 2003). Another interpretation is found in the *heuristics and biases* program in Psychology, where the use of heuristics by humans came to be viewed as leading to biases in their decisions (Tversky & Kahneman, 1974). Research in Psychology and other disciplines has shown that these interpretations do not match the original definition of bounded rationality and do not provide insights into the decision making process (Gigerenzer, 2016; Katsikopoulos & Gigerenzer, 2026).

Our work builds on the principle of *ecological rationality*, based on the original definition of bounded rationality and heuristics (Gigerenzer, 2020). It focuses on decision making under *open world uncertainty*, where the space of possible states and outcomes is not known in advance, the optimal solution is ill-defined, and probabilistic estimates may cease to be meaningful. The behavior of a human or an AI system is characterized as a joint function of the internal cognitive processes and the environment, with *adaptive satisficing* as the guiding principle for designing *simple heuristics*.

The difference between modern AI methods and simple heuristics can be understood in terms of the *bias-variance decomposition* of the expected squared prediction error (Geman et al., 1992). Having many tunable parameters can lead to overfitting; simple heuristics, on the other hand, are based on limited free parameters. They may lead to a higher bias component but still have lower overall error than modern AI methods because of the much lower variance component. Unlike modern AI methods that are largely *prescriptive*, decision heuristics are both prescriptive and *descriptive*, which supports the automatic generation of process-level descriptions as explanations of decisions. There is limited use of decision heuristics in architectures for robots or AI agents, except in the cognitive systems community (Langley & Katz, 2022) and prior work in our group (Dodampegama & Sridharan, 2026). In this work, we use heuristic methods to aid in the design and use of our compositional hierarchy.

3. Problem Formulation and Architecture

Consider an embodied AI agent assisting a human in a home environment to perform tasks $\{\tau_1, \tau_2, \ldots, \tau_n\}$ assigned sequentially. A contiguous subsequence of thematically related tasks forms a *session* (e.g., a sequence of commands constituting a routine for preparing for afternoon tea). Session boundaries define the scope of short-term contextual signals, e.g., salience evidence accumulates within a session and resets at each boundary. Each task τ_i is executed as a sequence of fine-grained actions such as *pickup*, *bring*, and *open*.

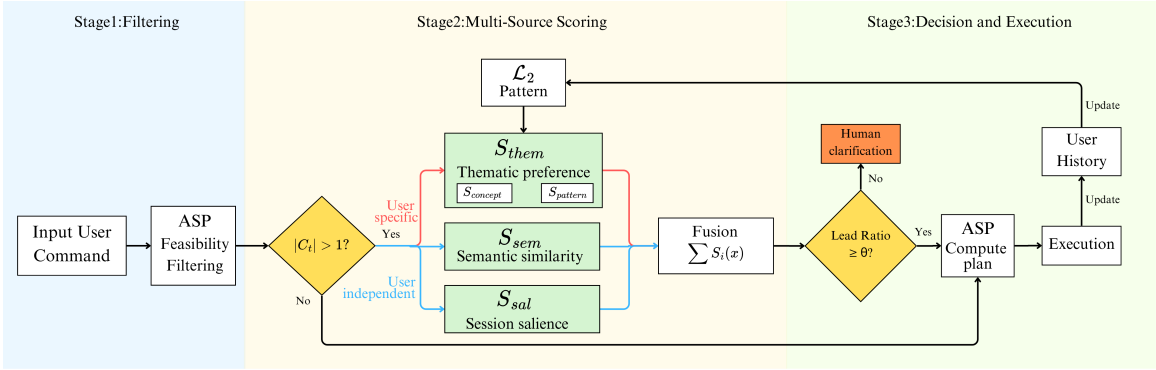


Figure 2. Three-stage disambiguation framework.

Disambiguation is required when: (a) the command contains an under-specified reference, such as a category term (e.g., "the drink") or a pronoun (e.g., "it"), that is compatible with multiple entities in the current state; or (b) the command is incomplete (e.g., "please fetch the . . . ") because the human input was not processed properly.

Figure 2 outlines the flow of information and control in our architecture, organized in three stages over two phases. In the first stage, the agent parses any given command to construct a candidate set $\mathcal{C}$ of feasible interpretations. This involves ASP-based non-monotonic logical reasoning (Section 3.1) with any prior knowledge (if available) of domain state and constraints to filter infeasible candidates. If there are multiple feasible candidates, they are then ranked in the second stage using three heuristically-modeled complementary signals (Section 3.3): (i) user-independent semantic compatibility derived from WordNet (Miller, 1995); (ii) session-level salience; and (iii) user-specific thematic preference estimated from interaction history (Section 3.2). In the third stage, if no candidate clearly resolves the ambiguity, the agent solicits clarification from a human. Once the agent identifies a specific interpretation, ASP-based reasoning is used to compute and execute the corresponding task plan. This trace of interaction and observations is also added to the user-specific history, enabling subsequent revision of the learned user preferences.

3.1 Knowledge Representation and Reasoning with ASP

In our architecture, any domain's transition diagram is represented using action language $\mathcal{AL}_d$ (Gelfond & Inclezan, 2013). The domain representation comprises a system description $\mathcal{D}$, a collection of statements of $\mathcal{AL}_d$, and a history $\mathcal{H}$. $\mathcal{D}$ has a sorted signature Σ with basic sorts, and the domain attributes (statics and fluents) and actions are described in terms of these basic sorts.

The basic sorts of our domain, and the three types of axioms (causal laws, state constraints, and executability conditions) that describe the domain's dynamics based on Σ, are illustrated in Appendix A. In addition to $\mathcal{D}$, the history $\mathcal{H}$ records the observations received (obs literals) and the actions executed (hpd literals) at specific time steps.

ASP-based non-monotonic logical reasoning is used for inference, planning and diagnosis. To perform such reasoning with knowledge, an ASP program $\Pi(\mathcal{D}, \mathcal{H})$ is automatically constructed (using a Python script), as detailed in Appendix A.

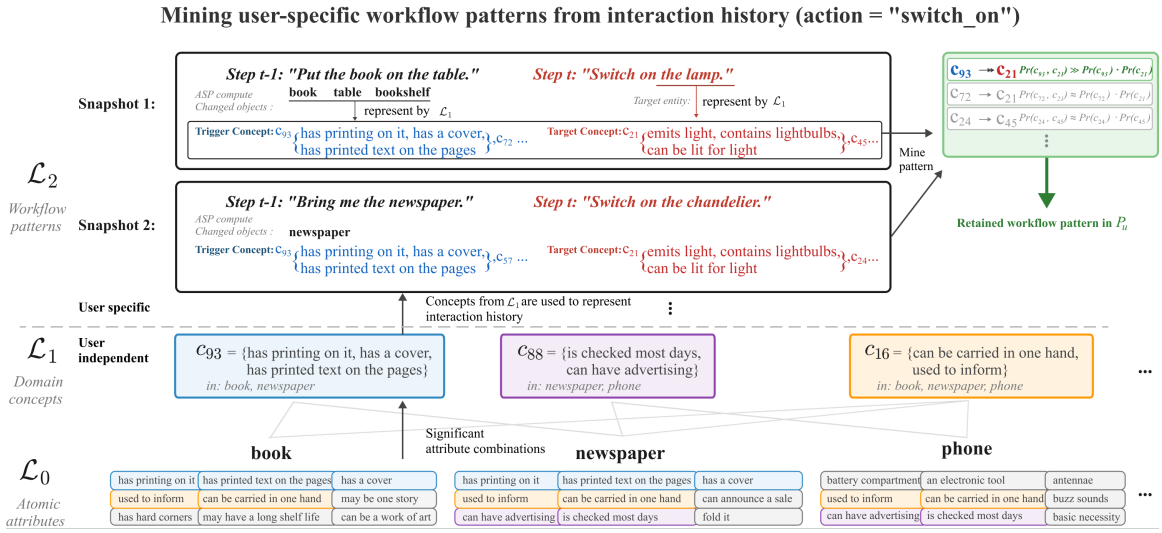


Figure 3. Overview of the three-layered hierarchical and compositional representation. Figure illustrates two example snapshots for action *switch on*, with the retained pattern $c_{93} \rightarrow c_{21}$ indicating that this user tends to interact with entities that emit light after interacting with printed reading material.

For inference and filtering of infeasible candidate interpretations of the user command (in Stage 1), the agent queries $\Pi(\mathcal{D}, \mathcal{H})$ to mentally simulate the execution of the command and compare the fluent values between current state and the resultant state. Any entity $o \in \mathcal{O}$ that appears as an argument of a fluent literal whose value has changed between the two states is identified and collected in a set O_t^{Δ} at time t. This set serves as contextual input for subsequent disambiguation (as described in Sections 3.2–3.3). All reasoning tasks are reduced to computing *answer sets* of Π using the SPARC system (Balai et al., 2013). For example programs corresponding to our paper, please see (Fu & Sridharan, 2026).

Although ASP filtering removes candidates violating known domain constraints, multiple feasible interpretations of the command often remain. Identifying and ranking these feasible candidates is difficult, particularly when considering domain-specific constraints and user preferences, since knowledge of the domain and the history ($\mathcal{H}$) of interactions involving any specific user may be limited. Simple heuristics are well-suited for making decisions in such circumstances. In our architecture, the use of heuristics is driven by the following two simple rules:

1. Describe any (new) object in terms of attributes commonly used (by humans) to describe previously known objects in the domain, and in terms of abstract concepts that capture attribute co-occurrences.
2. Identify the most likely object being referenced by matching attributes of candidate objects with those of known objects in recent interactions, including those involving this specific user.

While it is often claimed that FMs and deep networks capture statistics of object co-occurrence, it is difficult for such models to provide the desired personalization under uncertainty. Our work instead maps the first rule above to a simple compositional hierarchy of attributes and concepts that can be acquired and revised rapidly (Section 3.2). Based on this representation, our architecture

then maps the second rule to heuristic measures of *semantic similarity*, *salience*, and user-specific *thematic preferences* (Section 3.3). These measures are based on well-known simple heuristics such as the *recognition heuristic*, which advocates for choosing that which we know already over new entities (Goldstein & Gigerenzer, 2002), and the *weighted tallying heuristic* that combines a simple set of cues by adding evidence from each cue weighted by (learned) cue importance (Gigerenzer, 2016). We hypothesize and experimentally demonstrate that such an approach provides much better performance than modern AI models while requiring fewer resources.

3.2 Compositional Hierarchy of Attributes and Concepts

Since data needed to resolve ambiguities in complex domains is scarce, our architecture includes a three-layered hierarchical and compositional representation of objects in terms of their physical (appearance, functional) attributes:

- $\mathcal{L}_0$: domain objects (entities) are automatically represented in terms of fine-grained, shared, atomic attributes drawn from NOVA, an open-source semantic feature norm dataset (Suresh et al., 2026);
- $\mathcal{L}_1$: statistically significant attribute combinations are identified automatically to form a domain-level concept library $\mathcal{K}$, with entities then being represented in terms of elements of this concept library; and
- $\mathcal{L}_2$: entities are further represented in terms of user-specific workflow patterns and concept co-occurrences from records in $\mathcal{H}$ that correspond to actions (i.e., hpd literals) and match the current command.

Note that $\mathcal{L}_0$ and $\mathcal{L}_1$ are user-independent and constructed before deployment; they can be revised quickly over time but we keep them fixed in this paper. $\mathcal{L}_2$, on the other hand, is user-dependent, with each user maintaining a separate pattern set for each action (please see Figure 3 for examples).

$\mathcal{L}_0$: Atomic Attributes. Let $\mathcal{O}$ be the set of all entities in the domain. Each entity $o \in \mathcal{O}$ is represented as a set of atomic attributes $A(o) = \{a_1, a_2, \ldots, a_m\}$, capturing the properties that people regularly judge the entity to possess. These attributes are drawn from *semantic feature norms* in cognitive science, i.e., features collected by asking human participants to describe a concept's properties (McRae et al., 2005). We use NOVA (Suresh et al., 2026), a publicly-available dataset of human-validated feature norms covering 786 everyday concepts. Evidence can thus transfer across entities that share attributes that would be considered (by a human) to indicate similarity.

Attributes that are identified for only one entity are excluded, as they cannot contribute to cross-entity transfer of knowledge. The union $\mathcal{A} = \bigcup_{o \in \mathcal{O}} A(o)$ then defines the full vocabulary of atomic attributes for the domain. However, these shared attributes lack the selectivity required to isolate specific functional preferences, motivating the discovery of higher-level (composite) attributes.

$\mathcal{L}_1$: Domain-level Attributes. The next level in our hierarchy of attributes ($\mathcal{L}_1$) is a library $\mathcal{K}$ of statistically significant composite attributes, which we refer to as *concepts*. Concepts are identified automatically as combinations of attributes whose joint occurrence across the domain's entities is both statistically significant, i.e., the *lift* measure adapted from the data mining literature (Brin et al., 1997) exceeds a satisficing threshold λ_1, and supported by a minimum number of entities; attribute

pairs satisfying both criteria are incrementally extended to larger combinations (up to size five in our experiments). The formal criteria, thresholds, and examples are described in Appendix B.

In practice, a single entity is often represented by multiple concepts in $\mathcal{K}$, providing multiple pathways for evidence to transfer across structurally related entities. Conversely, attributes that do not participate in any retained combination are not covered by $\mathcal{K}$.

The final set $\mathcal{K}$ is a domain-specific, user- and category-independent set of concepts. In order to capture user-specific habits, the compositional hierarchy is extended to $\mathcal{L}_2$ by identifying workflow dependencies between these concepts.

$\mathcal{L}_2$: User-specific attributes. The next level of the concept hierarchy seeks to identify user-specific *habits*. For any given user, the agent accesses that user's separately maintained history $\mathcal{H}_u$ and extracts the records (i.e., hpd literals) corresponding to the same action as the current command. Each such record is then examined to obtain an *interaction snapshot*, i.e., two heuristically modeled sets of related concepts of relevant objects.

The *trigger concepts* are the $\mathcal{L}_1$ concepts of the entities impacted by the preceding command, i.e., of the entities in O^{Δ}_{t-1}; the *target concepts* are the $\mathcal{L}_1$ concepts $\Gamma(g_t)$ of the entity g_t that the agent acted on at step t, where $\Gamma(o) \subseteq \mathcal{K}$ denotes the set of $\mathcal{L}_1$ concepts describing entity o (see Equation 5 in Appendix B).

These snapshots capture the dependency between immediate world-state transitions and the subsequent user selections. Across all snapshots, the agent enumerates trigger-target concept pairs and retains only those whose co-occurrence is both frequent and statistically significant, applying the same 'lift' criterion with a relaxed threshold λ_2 to accommodate the smaller sample size for the current user. The retained pairs form the user-specific workflow pattern set P_u. A worked example of this construction, in the context of Figure 3, is presented in Appendix C.

3.3 Ranking Ambiguous Candidates and Disambiguation

The key part of Stage-2 of the pipeline of our architecture (in Figure 2) is the filtering and ranking of candidate interpretations $\mathcal{C}_t$ of the input user command. Our architecture uses an approach comprising three steps for this filtering and ranking.

Step 1: Filtering. As outlined in Section 3.1, the first step identifies candidates violating sort constraints in Σ or executability conditions in $\Pi(\mathcal{D}, \mathcal{H})$. After this filtering, if $|\mathcal{C}_t| = 0$, the agent requests clarification from the human; if $|\mathcal{C}_t| = 1$, this single candidate is selected immediately as the unambiguous interpretation and used for subsequent steps (e.g., planning and plan execution).

Step 2: Semantic similarity and salience. When $|\mathcal{C}_t| > 1$ after Step 1 above, each candidate $x \in \mathcal{C}_t$ is first scored by heuristic measures of semantic similarity and salience that consider recent contextual information but not the learned compositional hierarchy of attributes, i.e., information in $\mathcal{L}_1$ and $\mathcal{L}_2$ are not used. Specifically:

- *Semantic similarity* (S_{sem}): measures how well each candidate fits the context established by the preceding action, using a WordNet-based Wu–Palmer similarity score in $(0, 1]$.
- *Salience* (S_{sal}): measures how frequently each candidate has been the focus of attention in the recent past, within the current session.

Step 3: User-specific score. The thematic measure S_{them} estimates personalized preferences using the compositional concept hierarchy and the relevant records of this user's history, identified as described in Section 3.2. It is the sum of two heuristic measures:

- *Concept evidence* ($S_{concept}$): this gathers evidence of each candidate being the desired object, based on each $\mathcal{L}_1$ concept of the candidate and the user's recent interaction with objects with similar concepts using the action a under consideration. Since this signal can cease to be useful when the same action is applied equally to different types of entities (e.g., *switch_on* for lamp, microwave, television), we also consider the measure below.
- *Pattern evidence* ($S_{pattern}$): this gathers evidence in support of each candidate x based on set of $\mathcal{L}_2$ concepts for this user (i.e., P_u)—see Section 3.2. This evidence is the product of two factors: *follow rate* and *lift*, whose computation is illustrated with a worked example in Appendix C.

The thematic score is computed as the sum of the scores: $S_{them}(x) = S_{concept}(x) + S_{pattern}(x)$.

Combined Score and Selection. To combine these measures, we first make them comparable by normalizing each measure over the set of candidates ($\mathcal{C}_t$) to obtain a value $\in [0, 1]$. The individual measures are then fused by computing their weighted sum:

$$Score(x) \;=\; w_{sem}\, S_{sem}(x) \;+\; w_{sal}\, S_{sal}(x) \;+\; w_{them}\, S_{them}(x), \tag{1}$$

where the weights are experimentally determined from training data for each ambiguity level and shared by all methods that compute a match score. The normalized signals still differ in their values and purpose, e.g., salience concentrates on the recently mentioned candidate(s) whereas the thematic signal assigns similar weights to candidates that any particular user has interacted with. As discussed in Section 4.1, the weights encode the relative reliability of the signals, and the results are not sensitive over a range of values. The agent selects the top-ranked candidate only when its *lead ratio* over the second-ranked candidate, i.e., their score difference divided by the runner-up's score, is significant. Otherwise, the agent requests clarification from the human.

4. Experimental Setup and Results

We experimentally evaluated five hypotheses regarding our architecture's capabilities:

H1: The compositional hierarchy enables transfer of knowledge of shared concepts from previously-observed entities to entities with limited (or no) interaction history.

H2: Our architecture captures personalized preferences more effectively than LLM baselines that receive the same information, both with and without human-in-the-loop clarification, with the benefits increasing in more personalized scenarios.

H3: Aggregating statistics at the concept level ($\mathcal{L}_1$) improves disambiguation compared with statistics at the level of individual attributes ($\mathcal{L}_0$).

H4: Semantic compatibility, session salience, and user-specific thematic preference provide complementary evidence; their combination outperforms any factor considered individually.

H5: Concept-level statistics ($\mathcal{L}_1$) suffice under low levels of ambiguity, but user-specific patterns ($\mathcal{L}_2$) become more useful as ambiguity increases.

4.1 Experimental Setup

We created a household environment with 66 entities, four rooms, and 12 actions. Each entity is represented by its atomic attributes from the NOVA semantic feature norms dataset (Suresh et al., 2026) (Section 3.2), yielding an attribute vocabulary of $|\mathcal{A}| = 3{,}983$ across the 66 entities. Appendix D summarizes the concept-mining parameters and the resulting concept and pattern statistics.

The command set contains 200 high-level commands, each annotated with a textual description and an ASP-based goal. Based on input from five human participants (not involved in our architecture's design), we constructed and verified 10 interaction sessions per user, each containing 16 sequential commands under different initial conditions. For any given user, one of the 10 sessions was held out for testing and the remaining nine were used as historical information, with the held-out session excluded when computing S_{them}. We ran ten such trials per user, each holding out a different session, across all five users. Any given session comprises a coherent multi-step routine. In three sessions per user, the target objects are novel, and resolving them depends on transferring the user's preferences through concepts shared with familiar objects.

Ambiguity was generated by systematically replacing explicit object mentions with increasingly ambiguous references at four levels: (i) **A1** near hypernym (e.g., "apple"→"fruit"); (ii) **A2** higher hypernym ("apple"→"food"); (iii) **A3** broad hypernym ("apple"→"object"); and (iv) **A4** pronoun ("apple"→"it/that"). Each such reference corresponds to a sort in Σ (a pronoun may refer to any entity in the domain), and the corresponding argument of the command's ASP-based goal is replaced by a variable of this sort, e.g., the goal *has(user, apple)* becomes *has(user, X)* with X of sort $fruit$. The entities of this sort form the initial set of candidates $\mathcal{C}_t$, which is filtered and ranked as described in Section 3.3 to determine the value of X. For each session, we keep the first command unambiguous and apply ambiguities to the remaining commands, yielding 3,000 ambiguous commands in total (5 users $\times$10 sessions $\times$15 commands $\times$4 levels).

We report results under two modes: *noask*, where a method must always commit to the top-scoring candidate, and *ask*, where it may request clarification when evidence is insufficient. For the proposed method, clarification is triggered when the top candidate's lead over the runner-up falls below a confidence threshold (see Section 3.3). This threshold is set to be 0.25 at all ambiguity levels based on the observation that once all the scores have been computed and merged, there are $\leq$ 3–4 good candidates with a non-trivial score. Also, all methods that compute a match score (Equation 1 in Section 3.3), including the ablations, share the same fusion weights; the specific weight values and their robustness are discussed in Appendix D. For LLM baselines under *ask* mode, the model is allowed to output a clarification request when uncertain. We used GPT-5.1 in all LLM baselines.

4.2 Baselines and Performance Measures

We compared our architecture with eight baselines:

(B0) Random. One candidate is selected randomly after ASP-based filtering (Stage 1, Section 3.1).

(B1) LLM full-information + CoT. GPT-5.1 selects one candidate from the full set of potential candidates using chain-of-thought prompting (Wei et al., 2022), based on the current domain state, the current session, and the user's interaction history made available to our architecture.

(B2) LLM full-information + CoT, ASP-based filtering. This strategy builds on **B1**, with ASP-based feasibility filtering applied before the LLM selects from the remaining candidates.

(B3) Semantic and salience scores, ASP-based filtering. ASP-based feasibility filtering eliminates some candidates; S_{sem} and S_{sal} are then combined using Equation 1 and thresholds used in our architecture, but without user-specific thematic preference score (S_{them}).

(B4) Object-level history without compositional concepts, ASP-based filtering. This strategy uses the ASP-based filtering and the three scores of our architecture, but computes thematic preference over object categories instead of the hierarchical concepts.

(B5) $\mathcal{L}_0$ attributes, ASP-based filtering. This strategy builds on **B4**, but computes the thematic preference score over the raw $\mathcal{L}_0$ atomic attributes rather than object categories.

(B6) $\mathcal{L}_1$ concepts, ASP-based filtering. This strategy builds on **B4**, but the thematic score S_{them} is computed using concept-level evidence (based on $\mathcal{L}_1$) without the user-specific patterns ($\mathcal{L}_2$).

(B7) Full system, no ASP-based filtering. This strategy uses the three factors and compositional hierarchy for scoring the candidates, as in our architecture, but without ASP-based filtering.

Our architecture ("proposed") uses the ASP-based filtering and scores the remaining candidates by fusing three complementary signals: semantic similarity (S_{sem}), session salience (S_{sal}), and user-specific thematic preference (S_{them}) computed through compositional hierarchy of concepts (in $\mathcal{L}_0$, $\mathcal{L}_1$, $\mathcal{L}_2$). For paired comparison, each experimental run with our architecture was repeated for each baseline with the same users, sessions, ambiguous command instances, candidate sets, and history splits.

We used three performance measures: **Overall accuracy**, the proportion of commands for which the correct object was selected to address the ambiguity (in the *ask* mode, soliciting clarification from a human is considered as an incorrect outcome); **Answer accuracy**, similar to the previous measure but computed only over commands where the method made a definite selection, i.e., excluding trials with clarifications; and **Clarification rate**, the proportion of commands for which the approach being evaluated required clarification from a human to address the ambiguity.

4.3 Experimental Results

Appendix F presents execution traces that illustrate how the agent combines the three factors to resolve under-specified commands.

Overall comparison. Tables 1 and 3 (in Appendix E) summarize the results comparing our ("proposed") architecture with **B0–B7**. Our architecture led to significantly higher accuracy than all baselines at every ambiguity level (A1-A4) under both modes (*noask*, *ask*). We also observe that:

- Performance degrades the least with our architecture as ambiguity increases from A1 to A4. For example, the accuracy with our architecture drops from 74.2% to 54.7% in Table 1, but is still much better than any of the baselines; similar performance is observed in Table 3.
- The LLM-only baselines that do not use the compositional hierarchy or the simple heuristic measures (**B1–B2**) achieve substantially lower accuracy than our architecture despite receiving the same interaction history.

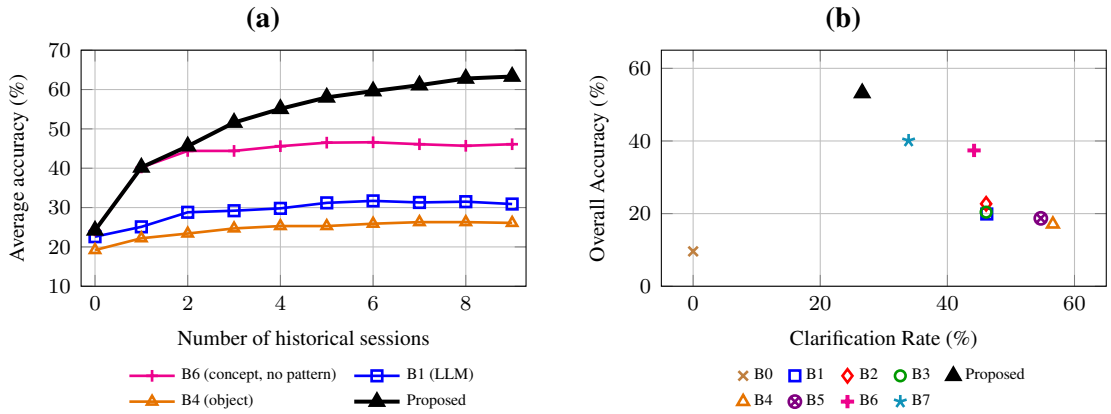


Figure 4. (a) Accuracy in *noask* mode (averaged over the four ambiguity levels) as a function of the number of historical sessions used to compute the user's patterns; (b) tradeoff between clarification and accuracy in *ask* mode; the ideal point on the upper-left, indicating high accuracy and minimal clarification.

- The ASP-based filtering is important for our architecture's performance; removing it (e.g., in **B7**) reduced accuracy substantially in Tables 1 and 3.
- Each layer of the compositional hierarchy contributes to our architecture's performance, e.g., accuracy drops substantially with increasing ambiguity without this hierarchy.

These results jointly provide evidence in support of all hypotheses **H1-H5**.

Knowledge transfer and impact of user-specific histories (H1-H2). We analyzed performance when test commands were grouped by how often the target object has appeared in recorded history: rare (≤ 5), medium (6–10), and frequent (11–20). We also explored how the performance changed with the amount and type of user-specific historical information made available to the AI agent, varying the number of historical sessions from 0 to 9 and computing the average accuracy across ambiguity levels. These experiments were conducted for the *noask* mode, with the corresponding results summarized in Table 2, Figure 4(a), and Table 4. We observed that:

- For rare targets, our architecture nearly tripled the accuracy of **B4** (no compositional hierarchy) and nearly doubled that of **B6** (no user-specific patterns)—see Table 2. This is due to the transfer of knowledge, through the concepts of the compositional hierarchy, from known objects to the new objects.
- Performance with baseline **B4**, which does not include the compositional hierarchy, remained almost unchanged as more historical information became available—see Figure 4(a)—because object identity does not transfer to novel entities. With our architecture, on the other hand, accuracy improved with additional user-specific histories.
- Computing S_{them} for a user from a different user's stored history performed worse than not using any user-specific history.

These findings provide strong support for hypotheses **H1-H2**.

Comparison with the LLM baselines (H2). We then analyzed the performance of our architecture in the context of the LLM baselines (**B1-B2**) that received the same historical information. Based on results in Table 1, Table 3 and Figure 4, we observe that:

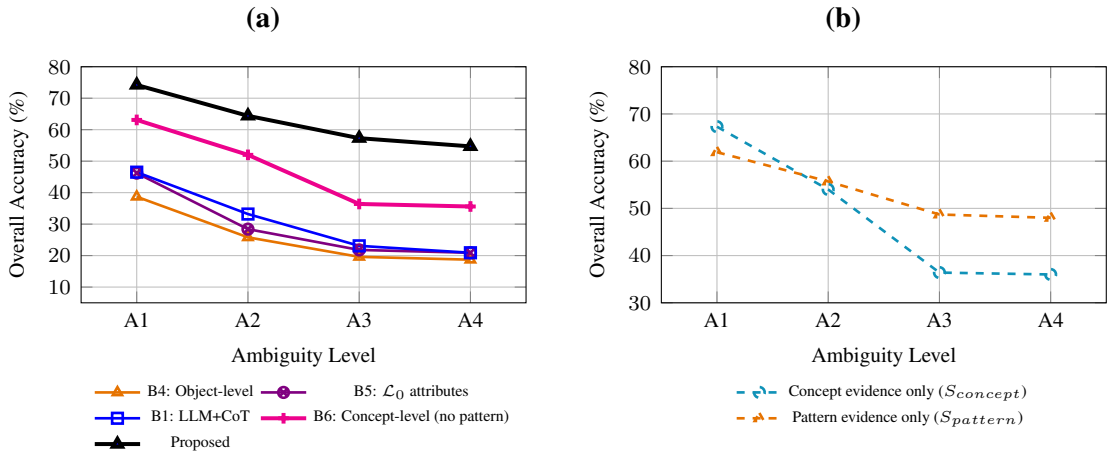


Figure 5. (a) Overall accuracy of some baselines and our architecture plotted as a function of ambiguity level in the *noask* mode; (b) overall accuracy plotted as a function of the ambiguity level in the *noask* mode when ASP-filtered candidates are ranked based on the heuristic measures that constitute thematic score (S_{them}).

- In the *noask* mode, the LLM baseline performed substantially worse than our architecture at each level of ambiguity, as shown in Table 1.
- In the *ask* mode, the LLM baselines requested clarification markedly more often than the proposed method, as summarized in Table 3. Even with these additional clarification queries, the LLM baselines provided much lower accuracy than our architecture.
- In Figure 4(a), accuracy for the LLM baselines increased only minimally with access to additional sessions of user-specific history, plateauing quickly when user-specific patterns are not modeled and leveraged explicitly. With our architecture, on the other hand, accuracy increased substantially as soon as some user-specific information is available.

These findings indicate the advantages of the compositional hierarchy and the ability to quickly and accurately model (and leverage) personalized user interactions; they thus strongly support **H2**.

Impact of compositional hierarchy and contextual factors (H3, H4). Next, we further analyzed the importance of the compositional hierarchy of concepts and the use of heuristically-modeled contextual factors as a function of the four ambiguity levels. Based on the results summarized in Table 1, Table 3, and Figure 5, we observed the following:

- Baseline **B3** is equivalent to removing the thematic factor (S_{them}), which considers the user-specific patterns, from our architecture; this decreased the average accuracy of our architecture substantially in Table 1. This is because these patterns provide crucial information in conjunction with the semantic similarity and salience scores.
- Using just the object identities with all three contextual factors and ASP-based filtering (**B4**) resulted in performance somewhat comparable to that of the LLM-based baseline in Figure 5(a). Also, the overall accuracy reduces substantially as the ambiguity level increases.
- Using just the atomic attributes level ($\mathcal{L}_0$) along with the contextual factors and ASP-based reasoning (**B5**) improved performance marginally compared with **B4**, but performance was still no

better than the LLM-based baseline (**B1**) in Figure 5(a) since the candidates for disambiguation often had considerable overlap of the atomic attributes.
- Baseline **B6** improved performance considerably compared with **B4-B5** and the LLM-based baseline (**B1**) in Figure 5(a).

Ranking the baselines based on performance leads to the order: objects $<$ attributes $<$ concepts $<$ concepts+patterns, which supports hypotheses **H3-H4**.

Contribution of user-specific workflow patterns (H5). Finally, we further analyzed the importance of acquiring and using the user-specific workflow patterns, and computing and using the thematic factor. The results led to the following observation:

- Figure 5(b) summarizes the accuracy when candidates are ranked based on the thematic factor alone, i.e., without the semantic and salience factors. Note that concept evidence ($S_{concept}$ by itself) provides better accuracy than pattern evidence ($S_{pattern}$) at a low ambiguity level (A1) based on near hypernyms. However, the situation reverses from ambiguity level A2, with user-specific patterns making a strong contribution at ambiguity levels A3–A4. Note that also adding workflow patterns ($\mathcal{L}_2$) raised the accuracy at every ambiguity level, with the largest gains at the higher ambiguity levels A3–A4—see Table 1 and Figure 5.

These results provide strong evidence in support of **H5**.

5. Conclusion

Foundation models and deep networks are increasingly considered state of the art for AI agents assisting humans, but they find it difficult to provide personalized responses in practical domains characterized by uncertainty and resource limitations. In a departure from methods that focus on the nature and type of training data, computational resources, or optimization methods used in existing monolithic AI models, we explored the design of an architecture for such AI agents based on some fundamental principles, focusing in particular on the principle of hierarchical compositionality and the use of simple heuristics. Overall, the quantitative and qualitative results demonstrate the clear performance gains obtained over state of the art LLM baselines, and the important contributions made by different components of our architecture based on the underlying principles.

Our architecture opens up multiple directions for further research that we will explore. First, we will support revising different layers of the compositional hierarchy over time, and expand it to include temporal concepts, i.e., changes in attributes over time. Second, we would like to implement and evaluate our architecture on a robot assisting different humans in indoor domains.

References

Abrini, M., & Chetouani, M. (2026). Clue: Crossmodal disambiguation via language-vision understanding with attention. *arXiv preprint arXiv:2602.08999*.

Abugurain, M., & Park, S. (2024). Integrating disambiguation and user preferences into large language models for robot motion planning. *arXiv preprint arXiv:2404.14547*.

Balai, E., et al. (2013). Towards answer set programming with sorts. *International Conference on Logic Programming and Nonmonotonic Reasoning* (pp. 135–147). Springer.

Beetz, M., et al. (2018). KnowRob 2.0 - A 2nd Generation Knowledge Processing Framework for Cognition-enabled Robotic Agents. *International Conference on Robotics and Automation*.

Boopathy, A., et al. (2024). Breaking neural network scaling laws with modularity. From `https://openreview.net/forum?id=unE3TZSAVZ`.

Brin, S., et al. (1997). Dynamic itemset counting and implication rules for market basket data. *Proceedings of the 1997 ACM SIGMOD International Conference on Management of Data* (p. 255–264). New York, NY, USA: Association for Computing Machinery.

Chen, W., et al. (2025). Neural probabilistic circuits: An overview. *Eighth Workshop on Tractable Probabilistic Modeling*.

Dodampegama, H., & Sridharan, M. (2026). Collaborate and Explain On-the-Fly: Knowledge-based Reasoning and Learning in Ad Hoc Teamwork. *Frontiers in Artificial Intelligence: Logic and Reasoning in AI*, *23*, 1–38.

Fidler, S., & Leonardis, A. (2007). Towards scalable representations of object categories: Learning a hierarchy of parts. *2007 IEEE Conference on Computer Vision and Pattern Recognition* (pp. 1–8). IEEE.

Fischer, G. (2001). User modeling in human–computer interaction. *User modeling and user-adapted interaction*, *11*, 65–86.

Fodor, J. A. (1975). *The language of thought*. Harvard University Press.

Fu, T., Jauw, B., & Sridharan, M. (2025). Combining llm, non-monotonic logical reasoning, and human-in-the-loop feedback in an assistive ai agent. *2025 34th IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)* (pp. 322–329). IEEE.

Fu, T., & Sridharan, M. (2026). Code for the compositional command disambiguation framework. `https://github.com/Tianyi-Fu/HCAA`.

Gao, P., et al. (2023). Compositional zero-shot learning for attribute-based object reference in human-robot interaction. *arXiv preprint arXiv:2312.13655*.

Gelfond, M., & Inclezan, D. (2013). Some properties of system descriptions of ald. *Journal of Applied Non-Classical Logics*, *23*, 105–120.

Gelfond, M., & Kahl, Y. (2014). *Knowledge representation, reasoning, and the design of intelligent agents: The answer-set programming approach*. Cambridge University Press.

Geman, S., et al. (1992). Neural networks and the bias/variance dilemma. *Neural Computation*, *4*, 1–58.

Gigerenzer, G. (2016). Towards a Rational Theory of Heuristics. In R. Frantz & L. Marsh (Eds.), *Minds, models and milieux: Commemorating the centennial of the birth of herbert simon*, 34–59. London: Palgrave Macmillan UK.

Gigerenzer, G. (2020). What is Bounded Rationality? In *Routledge Handbook of Bounded Rationality*. Routledge.

Goldstein, D. G., & Gigerenzer, G. (2002). Models of Ecological Rationality: The Recognition Heuristic. *Psychological Review*, *109*, 75–90.

Ivanova, A., et al. (2025). Ambik: Dataset of ambiguous tasks in kitchen environment. *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long*

Papers) (pp. 33216–33241).

Kambhampati, S., et al. (2024). Position: Llms can't plan, but can help planning in llm-modulo frameworks. *Forty-first International Conference on Machine Learning*.

Katsikopoulos, K., & Gigerenzer, G. (2026). Fast-and-Frugal Heuristics: Analytical Models of Intuition. *IMA Journal of Management Mathematics*, *37*, 17–33.

Koh, P. W., et al. (2020). Concept bottleneck models. *Proceedings of the 37th International Conference on Machine Learning* (pp. 5338–5348). PMLR.

Lampert, C. H., et al. (2013). Attribute-based classification for zero-shot visual object categorization. *IEEE transactions on pattern analysis and machine intelligence*, *36*, 453–465.

Langley, P., & Katz, E. P. (2022). Motion Planning and Continuous Control in a Unified Cognitive Architecture. *Annual Conference on Advances in Cognitive Systems*. Arlington, VA.

Lin, X., et al. (2025). Improving efficiency of answer set planning with rough solutions from large language models for robotic task planning. *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*.

McRae, K., Cree, G. S., Seidenberg, M. S., & McNorgan, C. (2005). Semantic feature production norms for a large set of living and nonliving things. *Behavior research methods*, *37*, 547–559.

Miller, G. A. (1995). Wordnet: a lexical database for english. *Communications of the ACM*, *38*, 39–41.

Mota, T., & Sridharan, M. (2021). Answer me this: constructing disambiguation queries for explanation generation in robotics. *2021 IEEE International Conference on Development and Learning (ICDL)* (pp. 1–8). IEEE.

Park, J., et al. (2023). Clara: classifying and disambiguating user commands for reliable interactive robotic agents. *IEEE Robotics and Automation Letters*, *9*, 1059–1066.

Piantadosi, S. T., et al. (2016). The Logical Primitives of Thought: Empirical Foundations for Compositional Cognitive Models. *Psychological Review*, *123*, 392–424.

Press, O., et al. (2023). Measuring and narrowing the compositionality gap in language models. *Findings of EMNLP 2023* (pp. 5687–5711).

Roy, S., et al. (2019). Leveraging past references for robust language grounding. *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)* (pp. 430–440).

Russell, S. J., & Norvig, P. (2003). *Artificial Intelligence: A Modern Approach (2nd ed.)*. Upper Saddle River, New Jersey: Prentice Hall.

Simon, H. A. (1956). Rational Choice and the Structure of the Environment. *Psychological Review*, *63*, 129–138.

Singh, A., et al. (2025). Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*.

Sridharan, M. (2025). Back to the future of integrated robot systems. *Proceedings of the AAAI Conference on Artificial Intelligence* (pp. 28610–28615).

Suresh, S., et al. (2026). Ai-enhanced semantic feature norms for 786 concepts. *Topics in Cognitive Science*, *18*, e70037.

Tian, Q., et al. (2024). Gsc: A graph-based skill composition framework for robot learning. *Robotics and Autonomous Systems*, *182*, 104787.

Tversky, A., & Kahneman, D. (1974). Judgement Under Uncertainty: Heuristics and Biases. *Science*, *185*, 1124–1131.

Wei, J., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*.

Appendix A. Domain Representation Details

This appendix presents the basic sorts and the example axioms of the domain representation described in Section 3.1.

Our domain includes basic sorts such as $room$, $container$, $furniture$, $food$, $human$, and $step$ (for temporal reasoning) that are arranged hierarchically, e.g., $fruit$ is a sub-sort of $food$ that is a sub-sort of $item$. Statics are domain attributes whose values cannot change, e.g., $next_to(kitchen, study)$, and fluents are attributes whose values can change. Fluents can be *inertial*, which obey inertia laws and are changed by actions, e.g., $loc(item, room)$ and $in_hand(agent, item)$; and *defined*, which do not obey inertia laws and are not directly changed by the agent's actions, e.g., $otherloc(human, room)$ is the human's location. Actions include the agent's actions, e.g., $move(agent, room)$, $pickup(agent, item)$, and $switchon(agent, appliance)$.

For any given domain, prior knowledge of the domain's dynamics is described based on the corresponding Σ using three types of axioms:

$$move(A, R) \textbf{ causes } loc(A, R) \tag{2a}$$

$$\neg\, at(A, R_1) \textbf{ if } at(A, R_2), R_1 \neq R_2 \tag{2b}$$

$$\textbf{impossible } give(A, O, U) \textbf{ if } loc(A, R_1),\ otherloc(U, R_2), R_1 \neq R_2 \tag{2c}$$

where Statement 2(a), a *causal law*, implies that when an agent (A) moves to a room R, its location becomes R; Statement 2(b), a *state constraint*, implies that an agent (A) cannot be in two places (R_1, R_2) at the same time; and Statement 2(c), an *executability condition*, prevents the agent (A) from trying to give an object (O) to a human user (U) who is not in the same room as the agent.

Statements in the history $\mathcal{H}$ have the form $obs(fluent, boolean, step)$, representing observations received at specific time steps, and $hpd(action, step)$, representing actions executed at specific time steps.

The ASP program $\Pi(\mathcal{D}, \mathcal{H})$ includes statements from $\mathcal{D}$ and $\mathcal{H}$, inertia axioms, reality check axioms, closed world assumptions for defined fluents and actions, and helper relations to reason over time steps, e.g., $holds(fluent, step)$ and $occurs(action, step)$ imply (respectively) that a fluent is true and that an action is part of a plan at a particular time step.

ASP is based on stable model semantics, and encodes *default negation* and *epistemic disjunction*; unlike classical negation "$\neg p$", which states that *p is believed to be false*, default negation "$not\ p$" only implies that *p is not believed to be true*. Each literal is true, false, or unknown, and the agent only believes that which it is forced to believe. ASP supports non-monotonic reasoning, i.e., the ability to revise previously held conclusions, which is essential for agents operating with incomplete knowledge and noisy observations.

For planning and diagnosis (in Stage 3), $\Pi(\mathcal{D}, \mathcal{H})$ includes helper axioms to define goals and guide the search for plans or explanations (diagnosis). After executing each action in the plan, the corresponding $hpd(action, step)$ and any resulting observations of the form $obs(fluent, boolean, step)$ are appended to $\mathcal{H}$ for subsequent reasoning.

Appendix B. Concept and Pattern Mining Details

This appendix presents the formal criteria used to mine the $\mathcal{L}_1$ concepts and the $\mathcal{L}_2$ workflow patterns described in Section 3.2. In our architecture, the process for identifying these concepts starts by considering pairs of attributes $\{a_i, a_j\} \subseteq A(o)$, where a_i and a_j may belong to the same or different dimensions, and retains pairs (i.e., compositions) that satisfy two heuristically-modeled criteria:

1. *Significance*: this criterion seeks to identify attributes whose joint occurrence is more significant than the occurrence of the individual attributes. To do so, we first focus on attribute pairs and adapt the *lift* measure defined in the data mining literature (Brin et al., 1997):

$$\text{lift}(a_i, a_j) = \Pr(a_i, a_j) \,/\, (\Pr(a_i) \cdot \Pr(a_j)) \geq \lambda_1 \tag{3}$$

   where λ_1 is an experimentally-determined satisficing threshold above which the joint occurrence of the attributes being examined is considered to be significant.

2. *Frequency*: this criterion seeks to identify attribute pairs that have support from a minimum number of entities in the domain. It is computed in the form of a simple count of objects (with the attribute pair under consideration) exceeding a threshold:

$$|\{o \in \mathcal{O} \mid \{a_i, a_j\} \subseteq A(o)\}| > \theta \tag{4}$$

   where θ is an experimentally-determined threshold. Together, these two criteria help identify attribute pairs shared across entities.

Next, the attribute pairs that pass both criteria are incrementally extended by considering additional attributes, with each such combination re-evaluated using the same criteria, conditioned on the already-confirmed subset. For example, $\text{lift}(a_k \mid \{a_i, a_j\}) = \Pr(a_i, a_j, a_k) \,/\, (\Pr(a_i, a_j) \cdot \Pr(a_k)) \geq \lambda_1$. This process continues until no further attribute passes both criteria or a predefined maximum size is reached for the concept. In our experiments, we set this upper bound on the size of each concept to five to limit computational cost while also identifying useful concepts.

This yields concepts of varying sizes (e.g., from pairs such as {*has_printing_on_it, has_printed_text_on_the_pages*} and triples such as {*has_printing_on_it, has_printed_text_on_the_pages, has_a_cover*}), each representing a more specific archetype to be considered for subsequent analysis.

Each interaction snapshot described in Section 3.2 comprises the following two heuristically modeled sets of concepts.

- *Trigger concepts*: this is the set of concepts (i.e., combinations of attributes) of objects that were impacted by the commands similar to current command. Specifically, for each entity $o \in \mathcal{O}$, a set of objects of interest, let $\Gamma(o) \subseteq \mathcal{K}$ denote the set of $\mathcal{L}_1$ concepts describing o, i.e., those concepts whose constituent attributes are all contained in o's attribute set $A(o)$. We can then describe the trigger concept set as:

$$\bigcup_{o \in O^{\Delta}_{t-1}} \Gamma(o), \quad \text{with} \quad \Gamma(o) = \{c \in \mathcal{K} \mid c \subseteq A(o)\} \tag{5}$$

where O^{Δ}_{t-1} is the set of entities that appear as an argument of a fluent literal whose value changed by the action executed by the agent at time $(t-1)$, and this change in value may have *triggered* the execution of the action at time t.

- *Target concepts*: this is the set of $\mathcal{L}_1$ concepts $\Gamma(g_t)$ describing the entity g_t (ground target object) that the agent acted on at step t in the historical record under consideration.

Appendix C. Signal Computations and Extended Examples

At the end of the $\mathcal{L}_0$ construction process (Section 3.2), the attributes are fine-grained and shared, e.g., *emits_light*, *contains_lightbulbs*, and *can_be_lit_for_light* appear across the lamp and the chandelier, while *has_printed_text_on_the_pages*, *has_printing_on_it*, and *can_be_carried_in_one_hand* appear across the book and the newspaper.

The following example illustrates the construction of the user-specific pattern set P_u ($\mathcal{L}_2$ in Section 3.2). In the context of Figure 3, suppose a user's interaction history $\mathcal{H}$ contains multiple records of the execution of the action *switch_on*, and suppose that multiple entities in the domain can be switched on, e.g., the lamp, the chandelier, and the television. In one record, the preceding instruction "Put the book on the table" changed the state of the book, the table, and the bookshelf, and the union of the corresponding concepts (e.g., $c_{93}\{$*has_printing_on_it, has_printed_text_on_the_pages, has_a_cover*$\}, \ldots$) forms the trigger set. The user then interacted with the lamp, whose concepts (e.g., $c_{21}\{$*emits_light, contains_lightbulbs, can_be_lit_for_light*$\}, \ldots$) form the target set. In another record, the trigger set derived from "Bring me the newspaper" again includes c_{93}, and the selected chandelier again includes c_{21}. Across snapshots, the pair of concepts (c_{93}, c_{21}) co-occurs significantly more than chance, i.e., $\Pr(c_{93}, c_{21}) \gg \Pr(c_{93}) \cdot \Pr(c_{21})$, and is retained in P_u.

For the semantic similarity (S_{sem}): the agent uses predefined associations between entities associated with the preceding action and the abstract context categories (e.g., *dining*, *reading*, *relaxing*), weighted by association strength. Summing these weights across the entities in O^{Δ}_{t-1} and taking the highest-scoring category yields a single context keyword w_{ctx} based on the preceding action. The Wu–Palmer similarity between each candidate $x \in \mathcal{C}_t$ and w_{ctx} in WordNet then gives a score per candidate, which is normalized across $\mathcal{C}_t$ to sum to one.

For the salience (S_{sal}): each candidate $x \in \mathcal{C}_t$ accumulates geometrically decayed weights over the recent action history, with more recent occurrences contributing more, and the resulting scores are normalized across $\mathcal{C}_t$ to sum to one.

Let us consider, again, the example above (from the description of level $\mathcal{L}_2$), in which the instruction "Put the book on the table" has just been executed. Now suppose the user issues an under-specified command "switch it on," and Step 1 of Section 3.3 leaves three candidates: *lamp*, *microwave*, and *television*. Since the preceding action involves *book-like* entities predominantly associated with the *reading* context, $w_{ctx} =$ *reading* is identified as the single context keyword. Then, since *lamp* is found to be closest to this context (*reading*), it has the higher S_{sem} score among the three candidates. Independently, the S_{sal} measure favors entities most recently referenced in the current session; assuming that the lamp was referenced in the recent past, we once again have *lamp*

receive the highest S_{sal} value among the three candidates. Based on both measures, *lamp* is ranked first among the three candidates based on the domain-specific measures.

For the concept evidence ($S_{concept}$): for a candidate x, the agent starts with a score of 0 and identifies each of its concepts $c \in \Gamma(x)$. It also considers all past records of action a in the user's history ($\mathcal{H}_u$), and computes the proportion of these records that involve the concept c. This proportion is added to the score of candidate x, and the process is repeated for all entries in $\Gamma(x)$. For example, if the user has habitually *eaten* apples, apple-related concepts, e.g., {*sweet, crisp, can be red colored, ...*}, will dominate the past *eat* actions, so a candidate sharing more of them receives a higher $S_{concept}$ in the context of the eat action.

For the pattern evidence ($S_{pattern}$): for each candidate x, the agent starts with a score of 0 and accumulates evidence based on each pattern $pt \in P_u$ whose target concept c_{target} belongs to $\Gamma(x)$ and whose trigger concept $c_{trigger}$ is part of the previous command. This evidence is the product of two factors: *follow rate* and *lift*, where $\mathrm{lift}(c_{trigger}, c_{target})$ is computed as in Equation 3, with probabilities $\Pr(c_{trigger}, c_{target})$, $\Pr(c_{trigger})$ and $\Pr(c_{target})$ estimated from this user's history $\mathcal{H}_u$. The follow rate considers past records (for this user) in which the trigger concept appeared at step $t-1$, and computes the fraction of these records in which the target concept appeared at step t. Follow rate, by itself, can overemphasize the evidence from common targets, but lift (computed as above) can substantially emphasize small-sample coincidences. For instance, if a book-like concept appeared 20 times (in this user's history) and was followed by a lamp-like concept 12 times, the follow rate is $12/20 = 0.6$. Continuing with our example, when book-like and table-like concepts both appear in O^{Δ}_{t-1}, the agent looks up P_u and finds two matched patterns, both with a lamp-like target concept and evidence (computed as described above) of 0.55 and 0.30 respectively. $\Gamma(x)$ for the lamp candidate contains the lamp-like concept, the target of both patterns, so the candidate accumulates a value of $0.55 + 0.30 = 0.85$. $\Gamma(x)$ for the other two candidates, microwave and television, does not contain this concept; they accumulate a value of 0.

Appendix D. Additional Experimental Setup Details

This appendix provides additional details of the experimental setup described in Section 4.1. The average number of candidates per command is 6.0, 16.6, 52.2, and 66.0 (out of 66 entities) for A1–A4 respectively. Commands in each session are processed sequentially. After each command, the symbolic world state is updated with the ground-truth referent to isolate errors in subsequent steps. We mined $\mathcal{L}_1$ concepts by retaining attribute combinations of size 2–5 that co-occur in at least 10% of the entities and satisfy the lift threshold $\lambda_1 = 2.0$, producing $2{,}508$ concepts mapped to the 66 entities. For $\mathcal{L}_2$ workflow pattern mining, we set $\lambda_2 = 1.2$ with a minimum support of two snapshots; each user's history yields 138–150 interaction snapshots from which these patterns are mined.

The average number of candidates per command grows across the ambiguity levels because the four levels weaken the evidence available for disambiguation to different extents: the near hypernym (A1) still constrains the entity type, whereas the pronoun (A4) carries no type information and resolution must rely almost entirely on the session context and the user's habits.

All methods that compute a match score share the same fusion weights. Specifically, we used the relative contribution of the individual methods to the known (correct) outcome in the training set to estimate these weights. Since different sessions are considered for the training set and the held-out testing set in each trial, the estimated values of the weights could differ. However, we observed some common trends. For example, the thematic measure always received the largest weight, and the weights were always within a narrow band: $w_{them} \in [0.45, 0.62]$ and $w_{sal} \in [0.13, 0.15]$, with w_{sem} taking the remainder. In addition, we observed that the performance was not very sensitive to the value of the weights as long as relative values of the weights were similar; the average accuracy (in the disambiguation task) varied by $\leq 1\%$ across a range of these values such as $w_{them} \in [0.55, 0.80]$, $w_{sal} \in [0.10, 0.15]$.

Any given session comprises a coherent multi-step routine (e.g., coffee preparation, relaxing in the living room, studying, cooking). Sessions involving the same user share consistent preferences, while sessions across users exhibit systematically different preferences to ensure that user-specific history is useful for accurate disambiguation. In three sessions per user, the target objects are novel, appearing in no other session and coming from categories adjacent to familiar ones, e.g., a mug where the user's routines feature cups, or an apricot where they feature peaches. When such a session is held out for testing, these objects have no direct history of their own, so resolving them depends on transferring any given user's preferences related to the concepts that the novel object shared with the objects familiar to the AI agent.

The 200 high-level commands include, e.g., rearranging objects in specific configurations and fetching specific objects, with ASP-based goals such as "has(user, book)" or "on(cup, table)". As with other such simulated domains, the agent has knowledge of the domain's state (i.e., full observability). To capture realistic variation in user habits, the five human participants were not involved in our architecture's design and were recruited to provide descriptions of their household routines and preferences.

Appendix E. Detailed Result Tables

This appendix reports the detailed comparisons of our architecture with the baselines under both modes, together with the target-frequency and history-source analyses discussed in Section 4.

Table 1. Comparison of our architecture with **B0–B7** under *noask* mode; (↑) indicates larger values are better, and numbers in boldface indicate that the corresponding method is significantly better (99% level of significance) than the others. Our architecture outperforms all baselines at every ambiguity level. The gap between **B6** and our architecture is largest at A3–A4, indicating that workflow patterns become more useful as ambiguity increases; the LLM-only baselines (**B1–B2**) achieve substantially lower accuracy despite receiving the same interaction history.

Ambiguity Level →	**B0: Random**	**B1: LLM+CoT**	**B2: +CoT+ASP**	**B3: S_{sem}+S_{sal}**	**B4: Object-level**	**B5: $\mathcal{L}_0$ attr.**	**B6: Concept-level**	**B7: No ASP**	**Proposed**
Accuracy A1 (%) ↑	27.6	46.5	54.0	39.1	38.7	46.2	63.1	62.2	**74.2**
Accuracy A2 (%) ↑	7.1	33.2	40.4	26.2	25.8	28.4	52.0	52.9	**64.4**
Accuracy A3 (%) ↑	2.2	23.1	26.3	20.9	19.6	21.8	36.4	45.8	**57.3**
Accuracy A4 (%) ↑	1.3	20.9	22.9	19.6	18.7	20.9	35.6	35.1	**54.7**
Average (%) ↑	9.6	30.9	35.9	26.4	25.7	29.3	46.8	49.0	**62.7**

Table 4. Accuracy of our architecture with different sources of history of users' interaction under *noask* mode. As before, (↑) indicates larger values are better, and numbers in boldface indicate that the corresponding method is significantly better than others (99% level of significance). The different options only change the history used to compute S_{them}. In *mismatched user* setting, each user is paired with a different user's history.

History Source	**A1** ↑	**A2** ↑	**A3** ↑	**A4** ↑	**Avg.** ↑
Matched user	**74.2**	**64.4**	**57.3**	**54.7**	**62.7**
Mismatched user	31.2	15.5	8.8	8.3	16.0
No user history (B3)	39.1	26.2	20.9	19.6	26.4

Table 2. Accuracy (%) under *noask* mode, grouped by how often the command's target entity occurs in the user's history; (↑) implies larger values are better, and numbers in boldface indicate that the corresponding method is significantly better (99% level of significance) than others. **B4** is effective only with frequent targets, while our architecture provides good accuracy even for rare targets.

Method	**Rare** (≤ 5) ↑	**Medium** (6–10) ↑	**Frequent** (11–20) ↑
B4: Object-level	16.2	19.6	50.6
B6: Concept-level (no pattern)	24.2	41.8	83.2
Proposed	**44.6**	**60.8**	**89.0**

Table 3. Comparison of our architecture with **B0–B7** under *ask* mode; (↑) implies larger values are better, (↓) implies that smaller values are better, and numbers in boldface indicate that the corresponding method is significantly better (99% level of significance) than the others. The LLM baselines without the compositional hierarchy or heuristics (**B1–B2**) solicit clarifications from a human much more than our architecture but are still less accurate in addressing ambiguity. Our architecture solicits clarifications the least, is the most accurate when it commits, and attains the highest overall accuracy at every ambiguity level.

Ambiguity Level →	**B0: Random**	**B1: LLM+CoT**	**B2: +CoT+ASP**	**B3:** S_{sem}+S_{sal}	**B4: Object-level**	**B5:** $\mathcal{L}_0$ **attr.**	**B6: Concept-level**	**B7: No ASP**	**Proposed**
Overall accuracy (%) *(a clarification counts as not completed)*									
A1 ↑	27.6	25.9	29.7	32.0	27.1	28.9	57.3	54.2	**66.7**
A2 ↑	7.1	19.7	22.5	24.4	19.1	20.9	38.2	43.6	**54.2**
A3 ↑	2.2	18.1	19.7	13.3	11.6	12.9	27.6	36.0	**47.6**
A4 ↑	1.3	16.0	18.8	12.0	10.7	12.0	26.7	26.7	**44.4**
Average ↑	9.6	19.9	22.7	20.4	17.1	18.7	37.4	40.1	**53.2**
Answered accuracy (%) *(over the commands each method chose to answer)*									
A1 ↑	27.6	64.5	70.1	49.8	49.4	50.3	80.7	75.4	**82.6**
A2 ↑	7.1	49.3	54.7	35.6	33.2	35.9	69.3	63.6	**72.7**
A3 ↑	2.2	27.2	31.2	29.7	27.8	29.6	58.6	55.6	**69.0**
A4 ↑	1.3	23.4	27.4	28.6	26.3	28.1	54.1	44.7	**66.3**
Average ↑	9.6	41.1	45.9	35.9	34.2	35.9	65.7	59.8	**72.7**
Clarification (%) ↓	–	46.2	46.1	46.1	56.6	54.7	44.2	33.9	**26.6**

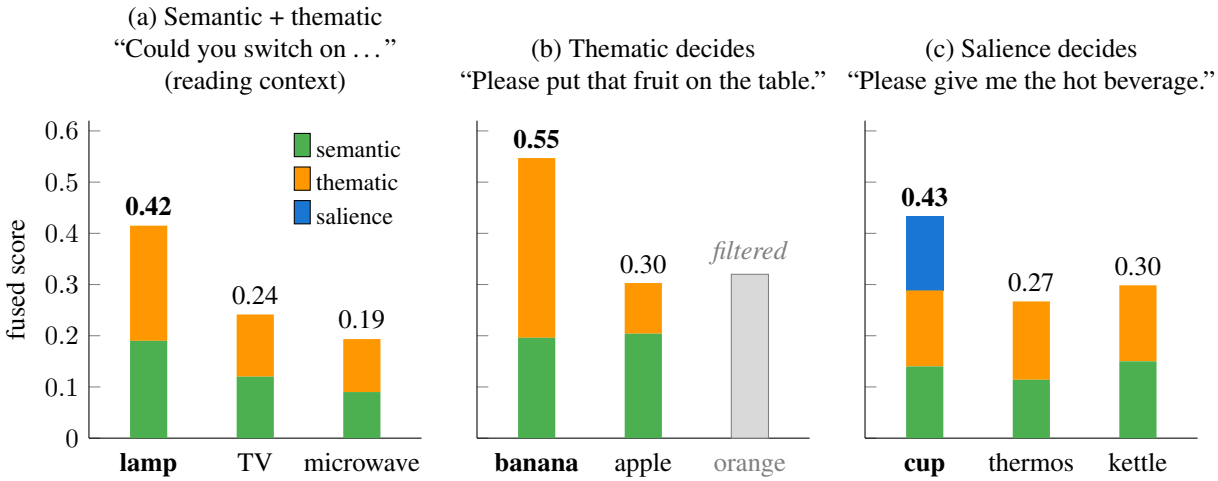


Figure 6. Fused per-candidate scores for the under-specified commands in each of the three execution traces in Sections F.1–F.3, decomposed into contributions from the three heuristically modeled contextual factors: semantic, salience, and thematic; the total score for the selected candidate is shown in boldface.

Appendix F. Execution Traces

We present three execution traces to illustrate how the agent combines the semantic compatibility, thematic preference, and session salience signals to resolve under-specified commands. Figure 6 reports the fused per-candidate scores for each command. The same three traces are illustrated in the supplementary video.

F.1 Execution Example 1: Fusion of contextual factors

Consider a living room that contains a bookshelf, a table, and three switchable appliances, a lamp, a television, and a microwave, with the agent in the same room.

- Earlier, the user had instructed the AI agent to: "`Please put the book on the table`", which the agent executed using the ASP-based plan:

  $$occurs(walk(agent1, living_room), 0),\ occurs(walktowards(agent1, bookshelf), 1),$$
  $$occurs(grab(agent1, book, bookshelf), 2),\ occurs(walktowards(agent1, table), 3),$$
  $$occurs(put(agent1, book, table), 4).$$

  to achieve *on(book, table)*. The objects changed by this command are recorded as O^{Δ}_{t-1}: *book*, *bookshelf*, and *table*.
- The user then speaks to the agent from the other end of the room, and the agent only hears "`Could you switch on ...`". ASP-based feasibility reasoning retains all three appliances *lamp*, *television*, and *microwave* as candidates.
- The recent action of placing the book on the table is associated with the reading context, identified using semantically relevant data from WordNet, so the *lamp* that shares properties with this context receives the highest semantic compatibility score.

- A user-specific workflow pattern mined from this user's history (Section 3.2), in which a book-like trigger concept is followed by a lamp-like target concept, gives the *lamp* the highest thematic preference score.
- No device was used recently, so session salience does not contribute. The two signals reinforce each other, and the fused score ranks the *lamp* first—see Figure 6(a). As the score for the *lamp* is sufficiently more than that of the second-ranked candidate, the agent commits to *holds(switched_on(lamp))* as the goal without requesting clarification. The corresponding trace of steps for computing the score for each candidate is shown in Figure 7.
- ASP-based reasoning then computes the plan:

$$occurs(walk(agent1, living_room), 0),\ occurs(walktowards(agent1, lamp), 1),$$
$$occurs(switchon(agent1, lamp), 2).$$

which the agent then executes to complete the task.

[COMMAND] "Could you switch on . . . "
[GOAL] switched_on(agent, switchable_item)
[FILTER] switchon accepts only sort #switch_furniture, 3 of 66 entities qualify
candidates = lamp, television, microwave
[CONTEXT] previous command = "put the book on the table", on(book, table)
changed = book, bookshelf, table
book reading 0.9, studying 0.5, relaxation 0.1
bookshelf reading 0.6, storage 0.5, studying 0.2
table dining 0.8, cooking 0.3, reading 0.1
totals: reading 1.6, dining 0.8, studying 0.7, storage 0.5, cooking 0.3
context word = reading
[SEMANTIC] Similarity to "reading" (WordNet)
lamp 0.475, television 0.300, microwave 0.225
the lamp fits a reading situation best
[THEMATIC] Concept evidence: past switch-on commands
lamp 0.08, television 0.50, microwave 0.43
Pattern evidence: trigger concepts of the previous command
c93 {has printing on it, has printed text on the pages, . . . }
c45 {has a flat surface, supports other objects, . . . }
target concept c21 {emits light, contains lightbulbs, can be lit for light}, carried by the lamp
lamp c93 to c21 0.55
c45 to c21 0.30
total 0.85
television, microwave: 0.00
combined: lamp 0.50, television 0.27, microwave 0.23
the mined pattern outweighs the concept evidence
[SALIENCE] nothing referenced this session
lamp 0.00, television 0.00, microwave 0.00
[SCORE] weights: semantic 0.40, thematic 0.45, salience 0.15

lamp	0.475	0.50	0.00	= 0.42
television	0.300	0.27	0.00	= 0.24
microwave	0.225	0.23	0.00	= 0.19

[DECIDE] lamp 0.42 over television 0.24, lead 0.75, threshold 0.25
committed without asking
[COMMIT] switched_on(lamp)

Figure 7. Trace of assigning factor values for each candidate considered for disambiguation in Execution Example 1.

F.2 Execution Example 2: Preferences from history of user interactions

Next, consider the AI agent assisting a human in a kitchen, in which an orange is already on the table, and an apple and a banana are within reach on the countertop.

- The human asks the AI agent to: "`Please put that fruit on the table`".
- ASP-based feasibility reasoning first removes the *orange* from further consideration because it is already on the table. This leaves the *apple* and the *banana* as candidates.
- The semantic compatibility score is almost the same for these remaining candidates, although there is a slight preference for the *apple*, and neither fruit has been handled recently by this user.
- The *banana*, however, shares concepts that recur in this user's records of this action, accumulated in the concept channel S_{concept}, so its thematic preference score is high enough to outweigh the difference in the semantic compatibility score, as shown in Figure 6(b). The corresponding trace of steps executed to compute the scores for each candidate is described in Figure 8.
- The agent then computes a plan to achieve the goal of placing the banana on the table, i.e., to achieve *on(banana, table)*. This is indeed the correct solution in this example, but would not have been identified without the explicit modeling and using of the user's history of similar interactions.

[COMMAND] "Please put that fruit on the table."
[GOAL] on(fruit, table)
[FILTER] orange dropped, on(orange, table) already holds
candidates = banana, apple
[CONTEXT] previous command = "put the orange on the table", changed = orange, table
context word = dining
[SEMANTIC] Similarity to "dining" (WordNet)
banana 0.49, apple 0.51
near tie, slight edge to the apple
[THEMATIC] Concept evidence: 98 past put commands

banana	c34 {has a soft interior, is soft on the inside, . . . }	31 of 98 = 0.32
	c41 {elongated, is cylindrical with a curved top, . . . }	24 of 98 = 0.24
	other concepts	0.19
	total	0.75
apple	c19 {crisp, crunchy, makes a crunchy sound, . . . }	8 of 98 = 0.08
	c23 {can be red colored, is sweet, is round, . . . }	5 of 98 = 0.05
	other concepts	0.08
	total	0.21

Pattern evidence: no pattern matched, 0.00 for both
combined: banana 0.78, apple 0.22
this user keeps acting on the concepts the banana carries
[SALIENCE] nothing referenced this session
banana 0.00, apple 0.00
[SCORE] weights: semantic 0.40, thematic 0.45, salience 0.15

banana	0.49	0.78	0.00	= 0.55
apple	0.51	0.22	0.00	= 0.30

[DECIDE] banana 0.55 over apple 0.30, lead 0.83, threshold 0.25
history outweighs the semantic edge of the apple
[COMMIT] on(banana, table)

Figure 8. Trace of assigning factor values for each candidate considered for disambiguation in Execution Example 2.

F.3 Execution Example 3: Use of salience as a key factor

In the third example, consider a kitchen in which a cup, a thermos, and a kettle stand together on the countertop, and the cup has just been filled from the kettle.

- The AI agent is asked by a human to: "`Please give me the hot beverage`".
- After basic validity checks, the AI agent reasons that all three objects: the *cup*, the *thermos*, and the *kettle*, are feasible candidates.
- The user does not show any thematic preference for any of these types of objects, so the semantic compatibility and thematic preference scores are similar for all three candidates.
- Session salience, computed from the maintained world state, assigns the highest weight to the most recently used candidate. Since the *cup* was just filled, it receives the highest (combined) score and is identified as the intended referent—see Figure 6(c) and the trace of steps to compute the scores in Figure 9.
- The AI agent then sets its goal to be: *has(user, cup)*, i.e., to ensure that the human user has the target cup. It then computes and executes a plan of actions to achieve this goal.

[COMMAND] "Please give me the hot beverage."
[GOAL] has(user, drinkware)
[FILTER] nothing dropped, all three can be handed over
candidates = cup, thermos, kettle
[CONTEXT] previous command = "fill the cup from the kettle", changed = cup
context word = dining
[SEMANTIC] Similarity to "dining" (WordNet)
cup 0.35, thermos 0.28, kettle 0.37
highest for the kettle, the wrong entity
[THEMATIC] Concept evidence: past give commands
cup 0.32, thermos 0.35, kettle 0.33
Pattern evidence: no pattern matched, 0.00 for all three
combined: cup 0.32, thermos 0.35, kettle 0.33
same concepts across the three vessels, no separation
[SALIENCE] recency this session, each step back discounted by 0.9

cup	filled one step back	0.90
thermos	not referenced	0.00
kettle	not referenced	0.00

relative: cup 1.00, thermos 0.00, kettle 0.00
[SCORE] weights: semantic 0.40, thematic 0.45, salience 0.15

cup	0.35	0.32	1.00	= 0.43
thermos	0.28	0.35	0.00	= 0.27
kettle	0.37	0.33	0.00	= 0.30

[DECIDE] cup 0.43 over kettle 0.30, lead 0.43, threshold 0.25
salience alone separates the three
[COMMIT] has(user, cup)

Figure 9. Trace of assigning factor values for each candidate considered for disambiguation in Execution Example 3.

Jointly, these execution traces demonstrate the working of our architecture and some of its capabilities.